

CVM++

Stack-Based Virtual Machine & Bytecode Compiler

Technical Report IIT Guwahati Coding Club, Even Semester 2026

Language	C++ (C++11 standard and above)
Build Command	g++ cvm.cpp -o cvm
Architecture	Zero-address stack-based VM
Category	C++ / Operating Systems / Compilers

1. Introduction

Building CVM++ teaches how a programming language actually works. It is a custom scripting language, bytecode compiler, and stack-based virtual machine. We have written it entirely in C++, with no external libraries. Raw source text goes in; results come out; everything in between is code I wrote myself.

2. System Architecture

The CVM++ pipeline consists of six discrete modules, each implemented as a self-contained C++ class, organised in a single translation unit (cvm.cpp) for simplicity of build and distribution.

2.1 Instruction Set Architecture (ISA)

The ISA is defined as a C++ enum of 14 opcodes stored as plain integers in a `std::vector<int>`. The choice of `int` rather than `uint8_t` keeps the implementation readable without a custom serialisation layer. The complete opcode table is:

Opcode	Operands	Semantics
OP_PUSH	1 immediate integer	Push literal onto stack
OP_ADD / SUB / MUL / DIV	none	Pop two operands, push result
OP_EQUAL / LESS	none	Pop two operands, push 1 or 0
OP_SET_VAR	1 slot index	Pop stack top into <code>globals[idx]</code>
OP_GET_VAR	1 slot index	Push <code>globals[idx]</code> onto stack
OP_JUMP_IF_FALSE	1 absolute IP target	Branch if top-of-stack == 0
OP_JUMP	1 absolute IP target	Unconditional branch
OP_INPUT	none	Read integer from stdin, push it
OP_PRINT	none	Pop and print top of stack
OP_HALT	none	Terminate the VM dispatch loop

2.2 Lexer (Scanner)

The Lexer class processes source code as a single `std::string` and exposes a single public method, `nextToken()`, that advances an internal character-position cursor and returns the next Token on demand. This on-demand (lazy) design avoids pre-allocating a token array and integrates naturally with the single-token lookahead the parser requires.

Tokenisation rules in order of priority: whitespace is silently skipped; digit sequences are accumulated into `T_NUM` integer literals; alpha/underscore sequences are matched against a keyword table (`let`, `print`, `input`, `true`, `false`, `if`, `else`, `while`) and otherwise emit `T_ID`; single- and double-character punctuation produces the corresponding operator or delimiter token. The `==` two-character operator is handled by peeking one character ahead.

2.3 Recursive Descent Parser

The Parser class implements a four-level precedence grammar:

- `parseFactor()` — atoms: numeric literals, boolean literals (`true/false` map to `1/0`), identifiers, input expressions, and parenthesised sub-expressions.
- `parseTerm()` — left-associative `*` and `/` applied to factors.
- `parseMath()` — left-associative `+` and `-` applied to terms.
- `parseExpression()` — left-associative `==` and `<` applied to math expressions.

Statement-level parsing (`parseStatement`) handles `let` declarations, reassignments, `print`, `if/else`, and `while`, delegating block bodies to `parseBlock` which consumes matching braces. The entry point `parseProgram` collects statements until `T_EOF`, producing a `ProgramNode` as the AST root.

2.4 AST Node Hierarchy

All AST nodes inherit from the abstract base `ASTNode` (with virtual `print` and virtual destructor). Node types are: `NumNode`, `VarNode`, `InputNode`, `BinOpNode`, `AssignNode`, `PrintNode`, `BlockNode`, `IfNode`, `WhileNode`, and `ProgramNode`. Ownership is managed entirely through `std::shared_ptr`, eliminating manual memory management. The `--show-ast` debug flag invokes the recursive `print()` tree-walk for visualisation.

2.5 Bytecode Compiler

The Compiler class walks the AST recursively via `dynamic_pointer_cast` dispatch and appends opcodes and operands to a public `std::vector<int>` bytecode. Key techniques:

- Variable slot allocation: an `unordered_map<string,int>` (`varMap`) assigns each identifier a monotonically increasing integer index on first encounter; subsequent references look up the same slot. Variables are stored and loaded from a `globals[256]` integer array in the VM.
- Forward jump patching for `if/else`: the compiler emits `OP_JUMP_IF_FALSE` with a placeholder operand, records that index, compiles the then-branch, emits `OP_JUMP` with another placeholder, then back-fills both placeholders with the correct absolute bytecode positions once the final sizes are known.
- Backward jump patching for `while`: the loop-start bytecode index is recorded before compiling the condition; after compiling the body, `OP_JUMP` is emitted with that saved index, and the forward exit jump is back-filled to the current end position.
- Expression compilation is post-order: operands are pushed before the operator opcode, matching the stack-based evaluation model of the VM.

2.6 Virtual Machine

The VM class holds the bytecode array, a 256-element raw `int stack[]`, a stack pointer (`stackTop`), and a 256-element `globals[]` array. The dispatch loop maintains an instruction pointer `ip` (`size_t`) and uses a switch statement for `O(1)` opcode dispatch:

- Arithmetic opcodes pop two operands, compute the result, and push it — three pointer-level operations per instruction.
 - `OP_JUMP_IF_FALSE` reads the branch target as the next bytecode word; it reassigns `ip` only if the popped condition value is zero, otherwise `ip` advances past the target word normally.
 - `OP_JUMP` unconditionally assigns `ip` to the encoded target.
 - `OP_INPUT` suspends the dispatch loop, reads a decimal integer from `stdin`, then pushes the value and resumes.
 - `OP_HALT` causes an immediate return from `run()`, cleanly terminating execution.
-

3. Build and Usage

CVM++ has no external dependencies beyond a standard C++11-compliant toolchain. Build with a single compiler invocation:

```
| g++ cvm.cpp -o cvm
```

Run a script file:

```
| ./cvm script.cvm
```

Optional debug flags:

- `--show-ast` — print the full indented Abstract Syntax Tree before compilation.
- `--show-bytecode` — print the raw integer bytecode array before VM execution.

Both flags may be combined. The script filename may appear anywhere in the argument list.

4. Test Scripts

Five canonical test scripts are provided, each targeting a specific subsystem:

File	Target Subsystem	What It Verifies
<code>arith.cvm</code>	Parser / Compiler	Operator precedence; parenthesised sub-expressions; multi-variable arithmetic.
<code>area_rect.cvm</code>	VM I/O + Variables	<code>OP_INPUT</code> suspend/resume; variable slot allocation; multi-statement programs.
<code>negative.cvm</code>	Signed Arithmetic	Stack handles negative 32-bit integers from subtraction; identity division.
<code>major_or_minor.cvm</code>	if/else Branching	<code>OP_JUMP_IF_FALSE</code> and <code>OP_JUMP</code> forward patch; correct else-block skip.
<code>loop.cvm</code>	while Loop	Backward jump to loop start; forward jump to exit; accumulator pattern.

5. Design Decisions & Trade-offs

- Single-pass, two-phase architecture: parsing and compilation are separate passes over the AST rather than a single-pass emit. This separation makes the code structure clear and easily extensible, at the cost of one additional tree traversal.

- `std::vector<int>` for bytecode: using `int` rather than `uint8_t` means the bytecode array is not compact (each opcode or operand occupies 4 bytes on most platforms), but it avoids byte-packing arithmetic and keeps operand reads trivial — a reasonable trade-off for a pedagogical project.
 - `shared_ptr` ownership: all AST nodes are reference-counted. This eliminates dangling-pointer bugs without requiring an arena allocator, and the AST is small enough that the reference-counting overhead is immaterial.
 - Fixed-size `globals[256]` and `stack[256]`: both arrays are statically allocated on the VM instance to avoid heap allocation overhead in the hot dispatch loop. A production VM would use a resizable structure; here the fixed size keeps the implementation accessible.
 - Absolute jump targets instead of relative offsets: `OP_JUMP` operands encode absolute bytecode indices. This simplifies jump patching (back-filling is a single integer assignment) and makes the bytecode dump straightforward to read, at the cost of slightly less portability if bytecode were ever serialised to disk.
-

6. Annotated Language Sample

The following loop accumulator script (`loop.cvm`) exercises the complete pipeline including variable allocation, while-loop backward and forward jump patching, and interactive input:

```
let loop_limit = input;           // slot 0
let current_count = 0;            // slot 1
let running_total = 0;            // slot 2
while (current_count < loop_limit) {
    let next_value = input;        // slot 3
    running_total = running_total + next_value;
    current_count = current_count + 1;
}                                  // backward OP_JUMP to loopStart
print running_total;
```

The compiler allocates four global variable slots (0–3), emits an `OP_JUMP_IF_FALSE` pointing past the loop body, emits all body instructions, then emits `OP_JUMP` pointing back to the condition start. The exit jump placeholder is back-filled after the entire loop body is compiled. This two-pass strategy within a single `compile()` call is the canonical solution to forward-reference resolution in single-pass code generation.

7. Conclusion

CVM++ is a complete, self-contained implementation of a language runtime built from first principles. Every component — scanner, parser, AST, compiler, and virtual machine — fits in approximately 473 lines of C++11 with no third-party dependencies. The codebase is small enough that each module can be understood in full, while the feature set is rich enough to compile and run real programs.